

Table of Contents

1) Introduction.....	1
2) Primary Data Types and OS allocation.....	2
a) Data type sizes across architectures.....	2
b) Practical example: observing data type sizes.....	3
3) OS Memory Segments.....	3
a) Memory Segment Overview.....	3
b) Memory Layout Visualization.....	4
4) Practical Code Examples.....	4
a) Observing variable addresses.....	4
b) Pointer and Memory Allocation.....	5\
5) Laboratory exercises.....	5
a) Lab 1: Data Type Size exploration.....	5
b) Lab 2: Memory Segment Address Mapping.....	7
6) Summary and Key takeaways.....	8
Knowledge check questions:.....	8

List of Tables

Listing 1. Program to Observe Data Type Sizes.....	4
Fig 1. Process Memory Layout (High to Low Addresses).....	5
Listing 2. Program to Observe Variable Addresses.....	6
Listing 3. Correct Pointer and Heap Usage.....	6

Lab 4: A Practical Approach to Understanding System Internals (Data Types and OS Memory Management)

1) Introduction

Data types form the foundation of programming and operating system interaction. A data type is a contract between

programmer, compiler, and operating system that specifies:

- The number of bytes to allocate in memory
- How to interpret those bytes (integer, character, floating - point)
- What operations are valid on that data

When a programmer declares a variable, the OS must allocate physical or virtual memory to store that data. Understanding this relationship is crucial for writing efficient, secure, and portable code.

2) Primary Data Types and OS allocation

a) Data type sizes across architectures

Table 1 shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

TABLE 1
DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

b) Practical example: observing data type sizes

```

GNU nano 8.7.1                                     datasize.c *
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:       %zu bytes\n", sizeof(int));
    printf("float:     %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));
    printf("long:      %zu bytes\n", sizeof(long));
    printf("pointer:   %zu bytes\n", sizeof(int*));

    return 0;
}

```

Listing 1. Program to Observe Data Type Sizes

3) OS Memory Segments

The operating system creates a process address space—a virtual view of memory divided into segments. Each segment serves a specific purpose and has different characteristics

a) Memory Segment Overview

TABLE 2
PROCESS MEMORY SEGMENTS AND THEIR PURPOSES

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

b) Memory Layout Visualization

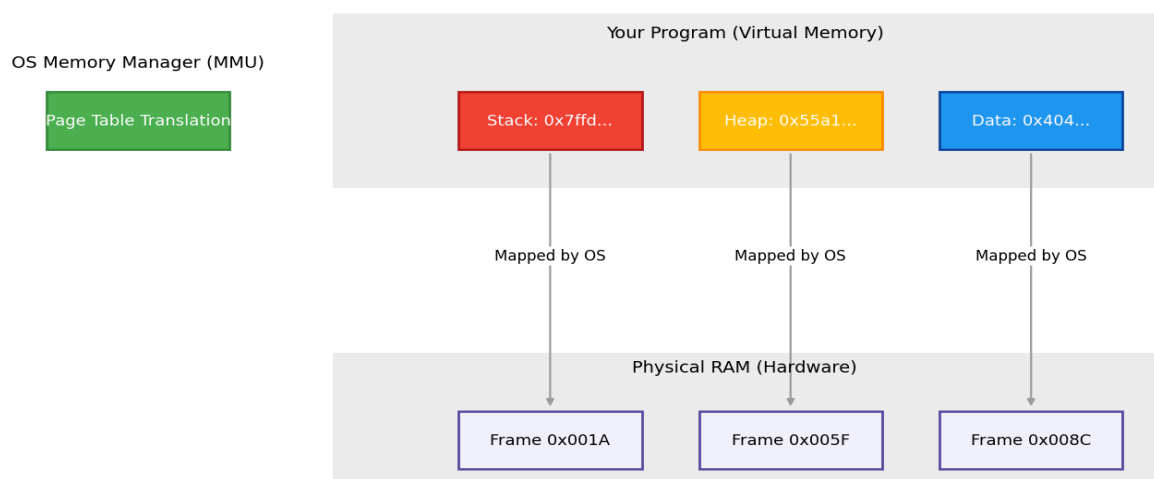


Fig 1. Process Memory Layout (High to Low Addresses)

Key Observation: Stack addresses are high (0x7ffd...), Heap addresses are mid-range (0x55a1...), and Data/BSS addresses are low (0x404...).

4) Practical Code Examples

a) Observing variable addresses

```
GNU nano 8.7.1 variableaddresses.c *
#include <stdio.h>
#include <stdlib.h>

int flobal_var = 150;    //Data segment
int bss_var;    //BSS/data segment

int main() {
    int local_var = 30;    //Stack
    static int local_static = 20; //Data segment

    int *heap_var = (int *)malloc(sizeof(int)); //Heap
    *heap_var = 500;
    printf(".....Variable Addresses.....\n\n");
    printf("global_var:    %p (Data/BSS) \n", (void *)&global_var);
    printf("bss_var:        %p (Data/BSS) \n", (void*) &bss_var);
    printf("local_var:         %p (Stack) \n", (void*) &local_var);
    printf("local_static:      %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr:       %p(Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var:        %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Listing 2. Program to Observe Variable Addresses

b) Pointer and Memory Allocation

```
GNU nano 8.7.1 pointer.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *forheap; // Declare pointer(lives on STACK )
    forheap = (int *)malloc(sizeof(int)); //allocate memory on heeap first!
    *forheap = 30; //store value to allocated memory
    //print addresses
    printf("Address of pointer (STACK) : %p\n", (void *)&forheap);
    printf("Address of data (HEAP) :      %p\n", (void *)forheap);
    printf("Value stored                %d\n", * forheap);

    return 0;
}
```

Listing 3. Correct Pointer and Heap Usage

5) Laboratory exercises

a) Lab 1: Data Type Size exploration

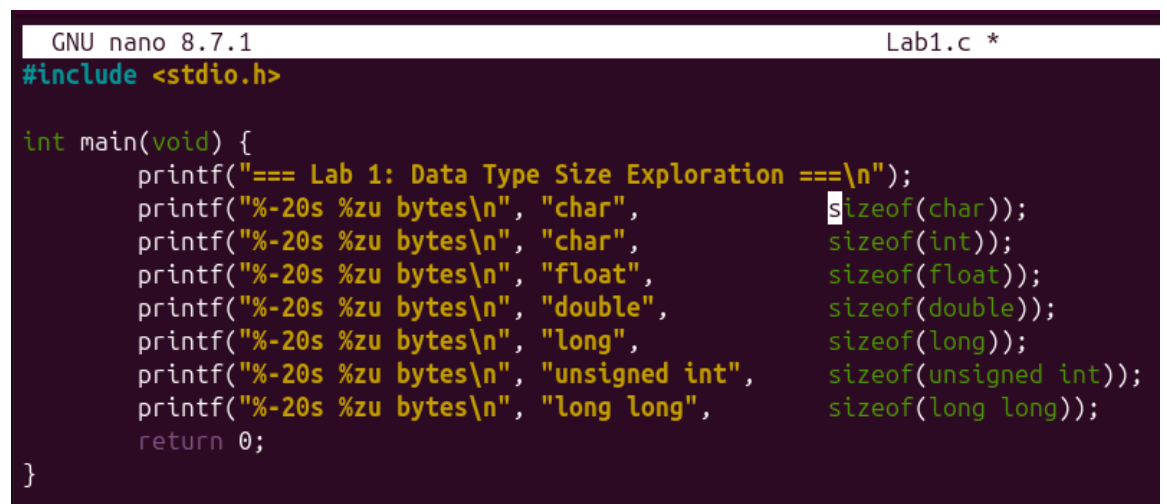
Objective: Use `sizeof()` to observe how much memory each data type uses on your system.

Tasks:

- i) Create a C program that prints the size of: char, int, float, double, long long
- ii) Compile and run on your system
- iii) Compare output with Table I
- iv) Answer: Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

Deliverables:

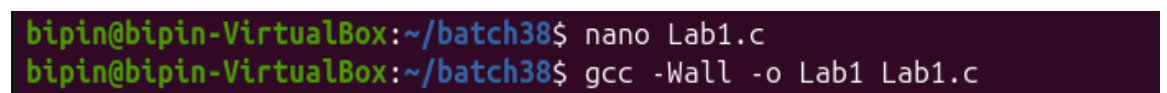
- Source code file (.c)



```
GNU nano 8.7.1 Lab1.c *
#include <stdio.h>

int main(void) {
    printf("=== Lab 1: Data Type Size Exploration ===\n");
    printf("%-20s %zu bytes\n", "char", sizeof(char));
    printf("%-20s %zu bytes\n", "int", sizeof(int));
    printf("%-20s %zu bytes\n", "float", sizeof(float));
    printf("%-20s %zu bytes\n", "double", sizeof(double));
    printf("%-20s %zu bytes\n", "long", sizeof(long));
    printf("%-20s %zu bytes\n", "unsigned int", sizeof(unsigned int));
    printf("%-20s %zu bytes\n", "long long", sizeof(long long));
    return 0;
}
```

- Screenshot of compilation (no errors)



```
bipin@bipin-VirtualBox:~/batch38$ nano Lab1.c
bipin@bipin-VirtualBox:~/batch38$ gcc -Wall -o Lab1 Lab1.c
```

- Screenshot of program output

```
bipin@bipin-VirtualBox:~/batch38$ ./Lab1
=== Lab 1: Data Type Size Exploration ===
char                1 bytes
int                 4 bytes
float               4 bytes
double              8 bytes
long                8 bytes
unsigned int         4 bytes
long long            8 bytes
```

- Why might sizeof(long) differ between 32-bit and 64-bit systems?
 - long sized to match the machine native word/pointer width, so it is typically 4 bytes on 32-bit systems and 8 bytes on 64-bit Linux/macOS.

b) Lab 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate OS memory layout.

Tasks:

i) Write a program that declares variables in each segment:

- Global initialized variable (Data segment)
- Global uninitialized variable (BSS segment)
- Local variable (Stack segment)
- Dynamically allocated variable (Heap segment)

ii) Print the address of each variable using %p

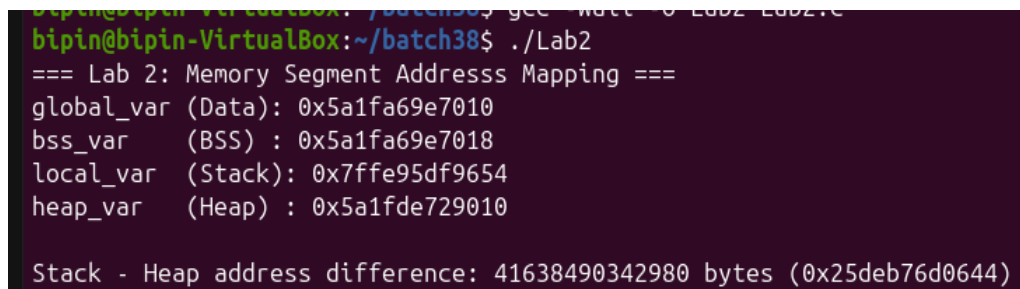
iii) Identify which segment has the highest addresses

iv) Identify which segment has the lowest addresses

v) Calculate the address difference between stack and heap

Deliverables:

- Screenshot of output



```
bipin@bipin-VirtualBox: ~/batch38$ ./Lab2
=== Lab 2: Memory Segment Addresss Mapping ===
global_var (Data): 0x5a1fa69e7010
bss_var     (BSS) : 0x5a1fa69e7018
local_var   (Stack): 0x7ffe95df9654
heap_var     (Heap) : 0x5a1fde729010

Stack - Heap address difference: 41638490342980 bytes (0x25deb76d0644)
```

- 2-3 sentences explaining what you observed
- The **stack** has the **highest addresses**, while the **Data and BSS** variables have the **lowest**, sitting next to each other near the program image. The **heap** sits above the Data/BSS segment and grows **upward**, while the **stack** grows **downward** from high memory. The two are separated by a huge unmapped gap, which lets both **grow without colliding**. The addresses change between runs because the **OS randomizes the layout**, but the **ordering stays the same**.

6) Summary and Key takeaways

- Data types are contracts—they tell the OS how much memory to allocate and how to interpret it.
- Memory is segmented—stack for locals, heap for dynamic, data/BSS for globals, text for code. (we will further explore the text section in more details if necessary)
- The OS manages it all—tracking, allocating, protecting, and virtualizing memory.
- Stack is automatic; heap is manual—choose based on lifetime and size needs.
- Virtual memory provides isolation—each process has its own address space.
- Pointers bridge stack and heap—the pointer variable lives on stack, but points to heap memory.

Knowledge check questions:

- 1) What is a data type in the context of programming and operating systems?
 - Defines the type of data a variable stores and how much memory it needs.
- 2) Name the five main memory segments in a process memory layout.
 - Text, Data, BSS, Heap, Stack
- 3) What is the difference between the Data segment and the BSS segment?
 - Data stores initialized global/static variables; BSS stores uninitialized ones (set to zero).
- 4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?
 - So stack can grow downward and heap can grow upward without colliding.
- 5) What happens to stack memory when a function returns? who reclaims it?
 - Stack frame is automatically removed; the system (OS) reclaims it.
- 6) How does the OS know which memory belongs to which process?
 - Using virtual memory and page tables.